

Correction — PF2 Partiel 4

Programmation Fonctionnelle 2 — juin 2026 — Pierre Rousselin
Université Sorbonne Paris Nord

Exercice 1 — Preuves de logique des propositions (12 pts)

1. Lemma ex4_1 : ((A $\vee$ B) $\rightarrow$ C) $\rightarrow$ (A $\rightarrow$ C) $\wedge$ (B $\rightarrow$ C) (4 pts)

Terme de preuve :

```
exact (fun h =>
  conj (fun hA => h (or_introl hA))
    (fun hB => h (or_intror hB))).
```

Avec tactiques :

```
Proof.
  intros h.
  split.
  - intros hA. apply h. left. exact hA.
  - intros hB. apply h. right. exact hB.
Qed.
```

Arbre de dérivation :

$\frac{[hA:A]}{\text{---}} \text{ (vI-g)}$ $\vdash A \vee B$ $h : (A \vee B) \rightarrow C$ $\text{---} \text{ (}\rightarrow\text{E)}$ $\vdash C$ $\text{---} \text{ (}\rightarrow\text{I sur hA)}$ $h \vdash A \rightarrow C$ $\text{---} \text{ (}\wedge\text{I)}$ $h \vdash (A \rightarrow C) \wedge (B \rightarrow C)$ $\text{---} \text{ (}\rightarrow\text{I sur h)}$ $\vdash ((A \vee B) \rightarrow C) \rightarrow (A \rightarrow C) \wedge (B \rightarrow C)$	$\frac{[hB:B]}{\text{---}} \text{ (vI-d)}$ $\vdash A \vee B$ $h : (A \vee B) \rightarrow C$ $\text{---} \text{ (}\rightarrow\text{E)}$ $\vdash C$ $\text{---} \text{ (}\rightarrow\text{I sur hB)}$ $h \vdash B \rightarrow C$ $\text{---} \text{ (}\wedge\text{I)}$ $h \vdash (A \rightarrow C) \wedge (B \rightarrow C)$ $\text{---} \text{ (}\rightarrow\text{I sur h)}$ $\vdash ((A \vee B) \rightarrow C) \rightarrow (A \rightarrow C) \wedge (B \rightarrow C)$
--	--

2. Lemma ex4_2 : A $\wedge$ $\sim$ A $\rightarrow$ B (4 pts)

Terme de preuve :

```
exact (fun h =>
  match h with conj hA hna => False_ind B (hna hA) end).
```

Avec tactiques :

```

Proof.
  intros [hA hna].
  exfalse.
  apply hna.
  exact hA.
Qed.

```

Arbre de dérivation :

$$\begin{array}{c}
 \frac{h:A \wedge \neg A}{\vdash A} \quad (\wedge E-g) \qquad \frac{h:A \wedge \neg A}{\vdash \neg A} \quad (\wedge E-d) \\
 \hline
 \vdash \text{False} \quad (\rightarrow E) \\
 \hline
 \vdash B \quad (\text{exfalse}) \\
 \hline
 \vdash A \wedge \neg A \rightarrow B \quad (\rightarrow I \text{ sur } h)
 \end{array}$$

3. Lemma ex4_3 : $\sim \sim \sim A \rightarrow \sim A$ (4 pts)

Terme de preuve :

```

exact (fun hnnn hA => hnnn (fun hna => hna hA)).

```

Avec tactiques :

```

Proof.
  intros hnnn hA.
  apply hnnn.
  intros hna.
  apply hna.
  exact hA.
Qed.

```

Arbre de dérivation :

$$\begin{array}{c}
 \frac{[hna:\neg A] \quad [hA:A]}{\vdash \text{False}} \quad (\rightarrow E) \qquad [hnnn:\neg \neg \neg A] \\
 \hline
 \vdash \neg \neg A \quad (\rightarrow I \text{ sur } hna) \\
 \hline
 \vdash \text{False} \quad (\rightarrow E) \\
 \hline
 \vdash \neg \neg \neg A \rightarrow \neg A \quad (\rightarrow I \text{ sur } hA) \\
 \hline
 hnnn \vdash \neg A \\
 \hline
 \vdash \neg \neg \neg A \rightarrow \neg A \quad (\rightarrow I \text{ sur } hnnn)
 \end{array}$$

Remarque : ce lemme illustre que si la double négation n'équivaut pas à l'affirmation en logique intuitionniste, la **triple négation**, elle, se simplifie toujours en une simple négation : $\neg \neg \neg A$ équivaut

à $\neg A$ (l'implication réciproque $\neg A \rightarrow \neg\neg A$ est également vraie et plus facile à montrer).

Exercice 2 — Arbres binaires colorés `ctree` (15 pts)

Type rappelé :

```
Inductive ctree (A : Type) :=
  | cleaf (key : A)
  | cnode (red : bool) (left right : ctree A).
Arguments cleaf {A}. Arguments cnode {A}.
```

1. Définition de `exple_5` (1 pt)

L'arbre donné (racine rouge, nœud interne gauche noir) :

```
      (rouge)
      /  \
    (noir) leaf 30
      /  \
    leaf 10 leaf 20
```

```
Definition exple_5 :=
  cnode true (cnode false (cleaf 10) (cleaf 20)) (cleaf 30).
```

2. Représentation graphique de `exple_6` (1 pt)

```
Definition exple_6 :=
  cnode false (cleaf "a") (cnode true (cleaf "b") (cleaf "c")).
```

```
      (noir)
      /      \
    leaf "a"  (rouge)
               /  \
            leaf "b" leaf "c"
```

3. Évaluation de `countRed exple_6` (1 pt)

```
Fixpoint countRed {A} (t : ctree A) : nat :=
  match t with
  | cleaf _      => 0
  | cnode r l r' => (if r then 1 else 0) + countRed l + countRed r'
  end.
```

Posons `R = cnode true (cleaf "b") (cleaf "c")` (le sous-arbre droit de `exple_6`, racine = false) :

```

countRed exple_6
= (if false then 1 else 0) + countRed (cleaf "a") + countRed R      [crN]
= 0 + countRed (cleaf "a") + countRed R
= 0 + 0 + countRed R                                              [crL]
= 0 + 0 + ((if true then 1 else 0) + countRed(cleaf "b") + countRed(cleaf "c")) [crN sur R]
= 0 + 0 + (1 + 0 + 0)                                             [crL, crL]
= 0 + 0 + 1
= 1

```

Résultat : 1

4. Preuve papier de flipColors_involutive (3 pts)

```

Fixpoint flipColors {A} (t : ctree A) :=
  match t with
  | cleaf a      => cleaf a                                (* fl *)
  | cnode r l r' => cnode (negb r) (flipColors l) (flipColors r') (* fn *)
  end.

```

Lemme : $\forall t, \text{flipColors} (\text{flipColors } t) = t$ — par induction sur t .

Cas cleaf a :

```

flipColors (flipColors (cleaf a))
= flipColors (cleaf a)      [fl]
= cleaf a                    [fl]
= t                          ✓

```

Cas cnode r l r', IHl : flipColors(flipColors l) = l, IHr' : flipColors(flipColors r') = r' :

```

flipColors (flipColors (cnode r l r'))
= flipColors (cnode (negb r) (flipColors l) (flipColors r')) [fn]
= cnode (negb (negb r)) (flipColors(flipColors l)) (flipColors(flipColors r')) [fn]
= cnode r (flipColors(flipColors l)) (flipColors(flipColors r')) [negb_involutive]
= cnode r l r'                                                    [IHl, IHr']
= t                                                                ✓

```

5. Preuve papier de length_toList (3 pts)

```

Fixpoint numLeaves {A} (t : ctree A) :=
  match t with
  | cleaf _      => 1                                (* nLl *)
  | cnode _ l r  => numLeaves l + numLeaves r      (* nLn *)
  end.

```

```

Fixpoint toList {A} (t : ctree A) : list A :=
  match t with
  | cleaf a      => [a]                                (* tLl *)

```

```
| cnode _ l r => toList l ++ toList r      (* tLn *)
end.
```

Lemme : $\forall t, \text{length} (\text{toList } t) = \text{numLeaves } t$ — par induction sur t , en utilisant `len_app`.

Cas cleaf a :

```
length (toList (cleaf a))
= length [a]          [tLl]
= 1
= numLeaves (cleaf a) [nLl] ✓
```

Cas cnode r l r', IHL : $\text{length}(\text{toList } l) = \text{numLeaves } l$, IHR' : $\text{length}(\text{toList } r') = \text{numLeaves } r'$:

```
length (toList (cnode r l r'))
= length (toList l ++ toList r')      [tLn]
= length (toList l) + length (toList r') [len_app]
= numLeaves l + numLeaves r'          [IHL, IHR']
= numLeaves (cnode r l r')           [nLn] ✓
```

6. Preuve papier de `iter_numLeaves` (3 pts)

```
Fixpoint iter {A B} (fl : A -> B) (fn : bool -> B -> B -> B) (t : ctree A) : B :=
  match t with
  | cleaf a      => fl a
  | cnode r l r' => fn r (iter fl fn l) (iter fl fn r')
  end.
```

Lemme : `iter (fun _ => 1) (fun _ x y => x + y) t = numLeaves t` — par induction sur t .

Cas cleaf a :

```
iter (fun _ => 1) (fun _ x y => x+y) (cleaf a)
= (fun _ => 1) a      [définition de iter, cas cleaf]
= 1
= numLeaves (cleaf a) [nLl] ✓
```

Cas cnode r l r', IHL, IHR' :

```
iter ... (cnode r l r')
= (fun _ x y => x+y) r (iter ... l) (iter ... r') [cas cnode]
= iter ... l + iter ... r'
= numLeaves l + numLeaves r'          [IHL, IHR']
= numLeaves (cnode r l r')           [nLn] ✓
```

Remarquons que le drapeau de couleur r est bien passé à `fn` mais n'est pas utilisé par `fun _ x y => x + y`, qui l'ignore : c'est cohérent puisque `numLeaves` ne dépend pas des couleurs.

7. Définitions avec iter (3 pts)

```
(* a) Nombre de nœuds rouges *)
Definition countRed' {A} (t : ctree A) : nat :=
  iter (fun _ => 0)
    (fun r x y => (if r then 1 else 0) + x + y)
    t.

(* b) Nombre de nœuds noirs (non rouges) *)
Definition countBlack {A} (t : ctree A) : nat :=
  iter (fun _ => 0)
    (fun r x y => (if r then 0 else 1) + x + y)
    t.

(* c) Toutes les feuilles vérifient p *)
Definition allLeaves {A} (p : A -> bool) (t : ctree A) : bool :=
  iter p (fun _ x y => x && y) t.
```

Exercice 3 — Prédicat inductif triple (13 pts)

```
Inductive triple : nat -> nat -> Prop :=
| tz : triple 0 0
| ts (n m : nat) : triple n m -> triple (S n) (S (S (S m))).
```

1. Règles d'inférence (1 pt)

_____ (tz)	Γ ⊢ triple n m
triple 0 0	_____ (ts)
	triple (S n) (S (S (S m)))

2. Preuves de triple26 et triple412 (4 pts)

a) triple 2 6 Arbre de dérivation :

```
_____ (tz)
triple 0 0
_____ (ts)
triple 1 3
_____ (ts)
triple 2 6
```

En Rocq :

```
Lemma triple26 : triple 2 6.
Proof. apply ts. apply ts. apply tz. Qed.
```

b) triple 4 12 Arbre de dérivation :

```

      _____ (tz)
triple 0 0
      _____ (ts)
triple 1 3
      _____ (ts)
triple 2 6
      _____ (ts)
triple 3 9
      _____ (ts)
triple 4 12
```

En Rocq :

Lemma triple412 : triple 4 12.

Proof. apply ts. apply ts. apply ts. apply ts. apply tz. Qed.

3. triple_three — induction sur n (4 pts)

Lemma triple_three (n : nat) : triple n (3 * n).

Proof.

induction n as [| n IH].

- (* n = 0 : Hypothèses : aucune. But : triple 0 (3*0) i.e. triple 0 0 *)
simpl. apply tz.
- (* n = S n', IH : triple n (3*n).
But : triple (S n) (3*(S n)) i.e. triple (S n) (S(S(S(3*n)))) *)
simpl.
apply ts.
exact IH.

Qed.

Justification du calcul : $3 * (S n) = 3*n + 3 = S(S(S(3*n)))$, ce qui correspond exactement à la conclusion du constructeur ts appliqué à IH : triple n (3*n).

4. triple_inv — induction sur la preuve (4 pts)

Lemma triple_inv (n m : nat) : triple n m -> m = 3 * n.

Proof.

intros H. induction H as [| n m Hnm IH].

- (* Cas tz : Hypothèses : aucune.
But : $0 = 3 * 0$ *)
reflexivity.
- (* Cas ts n m Hnm IH : Hypothèses : Hnm : triple n m, IH : m = 3*n.
But : $S(S(S m)) = 3 * (S n)$ *)
rewrite IH.
lia.

Qed.

Dans le second cas, IH : $m = 3*n$ permet de réécrire le but en $S(S(S(3*n))) = 3*(S\ n)$, qui est une égalité arithmétique « évidente » résolue par lia.

Exercice 4 — N, multiplication, listes (20 pts)

```
Inductive N : Set := N0 : N | Npos (p : positive) : N.
```

```
Definition Nmul (x y : N) : N :=
  match x, y with
  | N0, _           => N0
  | _, N0           => N0
  | Npos p, Npos q => Npos (Pos.mul p q)
  end.
```

```
Notation "x *N y" := (Nmul x y) (at level 40).
```

1. Valeurs de $N0 *N Npos\ xH$ et $Npos\ xH *N N0$ (1 pt)

```
N0 *N Npos xH = N0      (premier motif : N0, _ => N0)
Npos xH *N N0 = N0      (deuxième motif : _, N0 => N0)
```

2. Preuve de $Nmul_0_1$ (3 pts)

```
Lemma Nmul_0_1 (x : N) : N0 *N x = N0.
```

Aucune disjonction de cas n'est nécessaire : $Nmul\ N0\ x$ filtre sur le premier composant $N0$, et le premier motif $N0, _ \Rightarrow N0$ renvoie directement $N0$, quelle que soit la forme de x .

```
Proof.
  intros x.
  simpl.
  reflexivity.
Qed.
```

3. Preuve de $Nmul_1_1$ (5 pts)

```
Lemma Nmul_1_1 (x : N) : Npos xH *N x = x.
```

```
Proof.
  destruct x as [| q].
  - (* x = N0 : Nmul (Npos xH) N0 retombe sur le motif _, N0 => N0, et le but est N0 = N0 *)
    simpl. reflexivity.
  - (* x = Npos q : Nmul (Npos xH) (Npos q) = Npos (Pos.mul xH q) = Npos q par Pos.mul_1_1 *)
```



```

    simpl.
    rewrite Pos.mul_1_l.
    reflexivity.
Qed.

```

4. Définition de sumList (2 pts)

```

Fixpoint sumList (l : list N) : N :=
  match l with
  | []      => N0
  | h :: t => h +N sumList t
  end.

```

5. Preuve de prodList_app (9 pts)

```

Fixpoint prodList (l : list N) : N :=
  match l with
  | []      => Npos xH
  | h :: t => h *N prodList t
  end.

```

```

Lemma prodList_app (l1 l2 : list N) :
  prodList (l1 ++ l2) = prodList l1 *N prodList l2.

```

Proof.

```

  induction l1 as [| h t IH].
  - (* l1 = []
      But : prodList ([] ++ l2) = prodList [] *N prodList l2
      i.e. prodList l2 = (Npos xH) *N prodList l2 *)
    simpl.
    rewrite Nmul_1_l.
    reflexivity.
  - (* l1 = h :: t, IH : prodList (t ++ l2) = prodList t *N prodList l2
      But : prodList ((h::t) ++ l2) = prodList (h::t) *N prodList l2 *)
    simpl.
    rewrite IH.
    rewrite Nmul_assoc.
    reflexivity.

```

Qed.

Détail du second cas :

```

prodList ((h::t) ++ l2)
  = prodList (h :: (t ++ l2))
  = h *N prodList (t ++ l2)
  = h *N (prodList t *N prodList l2)      [par IH]
  = (h *N prodList t) *N prodList l2      [par Nmul_assoc]

```

`= prodList (h::t) *N prodList l2` ✓

Le premier cas réutilise directement le lemme `Nmul_1_l` prouvé à la question 3 : c'est exactement la justification de la propriété de neutralité de `Npos xH` pour `prodList []`.